

11

Data Hiding in Images

Virtually all sophisticated steganographic methods hide a message by embedding it as low-level noise in an image or audio file which then becomes the *cover file*. This approach has two disadvantages: the information hiding capacity of a cover file is small, so a large cover file is needed to hide a substantial amount of data; and once data are hidden in an image or audio file, any lossy compression destroys the embedded data. It seems that such an image should be compressed with lossless compression only, but this chapter shows how secret data can be hidden even in a lossily compressed image.

This chapter start with the description of methods for hiding data in the spatial domain of images. Starting with Section 11.8, several methods for transform-domain steganography are listed. Data hiding in binary images is the next topic, starting with Section 11.15.

11.1 LSB Encoding

Data can be hidden in a grayscale or color image, because slight changes to the colors of pixels are many times imperceptible to the eye (after all, changing the original pixels is the basis of all lossy image compression methods [Salomon 00]). The principle of LSB (least significant bit) encoding is to hide data bits by storing them in the least significant bits of the pixels of an image. The modified image is referred to as a stegoimage. It seems that the original idea of LSB encoding was due to Romana Machado [Stego 01].

Given a color image A with three bytes per pixel, each of the three color components, typically red, green, and blue, is specified by one byte, so it can have one of 256 shades. The data to be hidden are a stream of bits, each of which is hidden by storing it as the least significant bit (LSB) of the next byte of image A , replacing the original bit. Obviously, the least significant bit of the next byte of A is either a 0 or a 1. The

next bit to be hidden is also a 0 or a 1. This means that, on average, only half the least significant bits are modified by the bits being hidden. Each color pixel is represented by three bytes, so changing half the bytes means changing 1 or 2 least significant bits per pixel, on average.

Small changes in the color of a few isolated pixels may be noticeable if the pixels are located in large uniform areas or on sharply defined boundaries. It is therefore important to choose the cover image A carefully. It should contain a large number of small details with many different colors. Such an image is termed *busy*. An improvement to the basic LSB method is to hide a bit in a pixel only if the pixel satisfies certain conditions. If the variance of the luminances of the immediate neighbors of a pixel is very high (an indication that the pixel may be on a boundary) or very low (an indication that it is located in a uniform region), the pixel should be skipped. If a pixel passes these tests and is modified, it is important to verify that it passes the same tests after modification, since otherwise the decoder would not be able to reliably identify the pixels that have been skipped. If a pixel does not pass the tests after being modified, it should be returned to its original value.

A comparison of the original cover image and the stegoimage would show an attacker what bits have been modified and may serve as the basis of a successful attack on the hidden data. This is why the original image should be destroyed right after the stegoimage has been prepared. Another obvious precaution is to use the same cover image only once. If two modified versions of the same image fall into the wrong hands, they may provide precious clues for the hiding algorithm.

If image A is to be compressed later, the compression should, of course, be lossless. If the owner forgets this, or if someone else compresses image A with a lossy compression algorithm, some of the hidden data would be lost during the compression. Thus, this basic method is not robust.

The hiding capacity of this method is low. If one data bit is hidden in a byte, then the amount of data hidden is $1/8 = 12.5\%$ of the cover image size. As an example, only 128 K bytes can be hidden in a 1 Mbyte cover image. *Stego*, a widely available implementation of this method, is described in Section 12.10.1.

It is easy to come up with sophisticated variants of this method that use a stego-key and are therefore harder to attack successfully. Here are a few ideas.

1. If the key is, say, 1011, then data bits are hidden in the least significant bits of bytes 1, 3, and 4 but not of byte 2, then in bytes 5, 7, and 8, but not in byte 6, and so on.
2. Bits are stored in selected bytes as in variant 1, but are stored alternately in the least significant position and in the second least significant position of bytes of the cover image.
3. The stego-key is used as the seed of a pseudo-random number generator that generates a random sequence of small positive integers r_i . The image pixels are numbered P_i and bits are hidden in pixels $P_1, P_{1+r_1}, P_{1+r_1+r_2}$, and so on. Only certain pixels are affected, which reduces the data embedding capacity of the cover image. This variant is more secure but can be used for hiding only small quantities of information.
4. The stego-key is again used as the seed of a pseudo-random number generator and a random sequence of small positive integers r_i is generated. Data bits are hidden

in pixels as in variant 3, but when an index $1 + r_1 + r_2 + \dots + r_j$ exceeds the size of the image, it is computed modulo this size. The result is that data bits are hidden in pixels located farther and farther away from the start of the cover image, and following bits are hidden in bytes close to the start. It is easy to have a collision (a pixel that has already been modified is selected again), so this variant has to maintain a list of pixels that have been modified. When the algorithm arrives at such a pixel, the next pseudo-random number is drawn, and another pixel is selected for hiding the next bit. Notice that the decoder can mirror this operation.

5. Proceed as in the previous method but without maintaining a list of pixels that have been modified. Some of the embedded bits will be corrupted by bits stored "on top" of them, but these errors can be corrected by adding an error-correcting code to the data before they are hidden. The Hide and Seek software (Section 12.10.2) uses this method but with no error correction.

6. This variant is distantly related to BPCS steganography (Section 11.2). The stego-key is used to partition the cover image into disjoint regions. If the key is the single digit n , the image is partitioned into n equal-size strips (where the last strip may be shorter). If the key is the pair (r, c) , the image is partitioned into rectangles, each with r rows and c columns (where the rectangles on the right edge and on the bottom of the image may be smaller). The stego-key may be the seed of a pseudo-random number generator, and the random integers thus generated used to specify the regions.

A parity bit b_i is computed for each region i as the parity of the least significant bits of the region's pixels. One data bit d_i is hidden in the region. If b_i is different from d_i , then b_i is made equal to d_i by changing the least significant bit of one pixel in the region.

The decoder simply computes the parity bit of each region and concatenates the parity bits to form the string of hidden data.

Only a small amount of data can be hidden in this variant, but this method has the advantage that the encoder may be able to select, in each region, the pixel whose modification will change the region's statistics the least. A pixel that's very similar to its immediate neighbors should not be modified, since it is located in a uniform (or almost uniform) region, where any small change may be noticeable. Similarly, a pixel that's very different from its immediate neighbors should not be modified, since this may also be very perceptible. The encoder should therefore select a pixel in the following steps: (1) Compute the difference between a pixel and the average of its (four or eight) near neighbors. (2) Repeat for all the pixels in the region. (3) Let s and l be the smallest and largest differences. (4) Select a pixel whose difference is the closest to $(s + l)/2$.

7. No stego-key is used. This variant scans the image pixel by pixel along a complex path, not row by row, and hides one bit of the data in each pixel visited. For example, the pixels may be visited in the order defined by a space-filling curve, such as the Peano curve or the Hilbert curve (Section 2.1). Such curves visit each point of a square area exactly once.

8. The idea of this variant was published by [Aura 96] who also implemented it. A stego-key and a secure hash function H are used to generate a sequence of unique pixel addresses. One data bit is hidden in the LSB of each of those pixels. Hash functions are discussed in Appendix B. A secure hash function H has some useful properties: (1)

if $z = H(x)$, then it is computationally infeasible to find a $y \neq x$ such that $z = H(y)$, (2) collisions are extremely rare (i.e., it is also computationally infeasible to find two different arguments x and y that will hash to the same z). We assume that an image of dimensions $x \times y$ is given, where x is the horizontal dimension, i.e., the number of columns. A stego-key K is selected and is partitioned into three parts K_1 , K_2 , and K_3 . To find the image location (X, Y) where the i th data bit is to be hidden, compute the following (where $\circ$ and $\div$ indicate concatenation and integer division, respectively):

$$\begin{aligned} Y &= i \div x, \\ X &= i \bmod x, \\ Y &= (Y + H(K_1 \circ X)) \bmod y, \\ X &= (X + H(K_2 \circ Y)) \bmod x, \\ Y &= (Y + H(K_3 \circ X)) \bmod y, \\ \text{return } Y \times x + X. \end{aligned}$$

For example, assume that the image has dimensions 800×600 (again, 800 is the horizontal dimension, i.e., the number of columns) and the stego-key is the integer 123,456,789. To find the location where the 1001st data bit should be hidden, perform

$$\begin{aligned} Y &= 1001 \div 800 = 1, \\ X &= 1001 \bmod 800 = 201, \\ Y &= (1 + H(123 \circ 201)) \bmod 600 = (1 + 7377) \bmod 600 = 178, \\ X &= (201 + H(456 \circ 178)) \bmod 800 = (201 + 3854) \bmod 800 = 55, \\ Y &= (178 + H(789 \circ 55)) \bmod 600 = (178 + 1124) \bmod 600 = 102, \\ \text{return } Y \times x + X &= 102 \cdot 800 + 55. \end{aligned}$$

Thus, the value returned is $Y \times x + X = 102 \cdot 800 + 55 = 81,655$ or $(X, Y) = (55, 102)$. (End of variant 8.)

A marked improvement is achieved by considering the sensitivity of the eye to various colors. Of the three colors red, green, and blue, the eye is most sensitive to green and least sensitive to blue. Thus, hiding bits in the blue part of a pixel results in image modification that's less noticeable to the eye. The price for this improvement is reduced hiding capacity from 12.5% to 4.17%, since one bit is hidden in each group of three image bytes.

Figure 11.1a shows the well-known Lena image at a resolution of 128×128 . Figure 11.1b-d shows the results of embedding text in bits 3, 5, and 7, respectively, of each pixel. (Bit positions are numbered from 1, the least significant bit, to 8, the most significant one.) It is clear that embedding foreign bits in most significant positions can significantly degrade the quality of the cover image. Figure 11.2 lists simple Matlab code to embed text in an image.

How can we identify a given image as a cover image? Figure 11.1e,f shows the distributions of pixel values (in the range $[0, 255]$) of the same Lena image before and



(a)



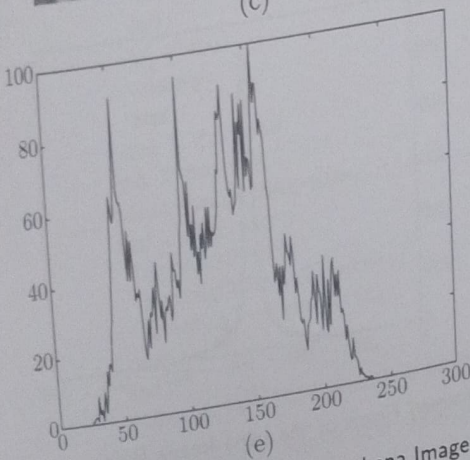
(b)



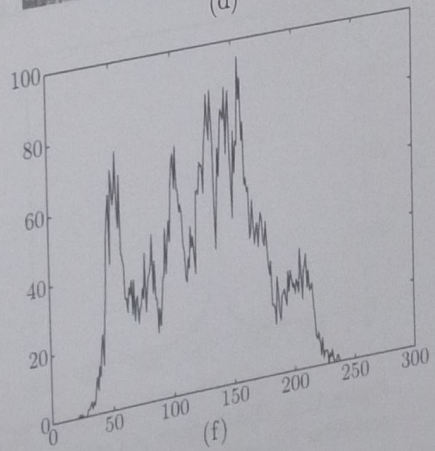
(c)



(d)



(e)



(f)

Figure 11.1: Lena Image and Pixel Distributions.

```

% Embed bits of text 'herring' in any bit position
% of pixels of image 'lena'
gmap=gray(256); % colormap for 8-bit pixels
filename='lena128'; dim=128;
fid=fopen(filename,'r');
if fid==-1 disp('image file not found'); end;
img=fread(fid,[dim,dim]);
status=fclose(fid);
figure(1), image(img), colormap(gmap), axis off, axis square
filename='herring';
fid=fopen(filename,'r');
if fid==-1 disp('data file not found'); end;
[hide,count]=fread(fid,[dim,dim],'ubit1');
count; % input text file bit by bit into 'hide'
status=fclose(fid);
pdist=zeros(256,1);
for row=1:dim
    for col=1:dim
        img(row,col)=bitset(img(row,col),1,hide(row,col));
    % 2nd param of bitset is bit position to embed data
        p=img(row,col)+1;
        pdist(p)=pdist(p)+1;
    end
end
figure(2), image(img), colormap(gmap), axis off, axis square
figure(3), plot(pdist) % dist of pixel values in embedded image

```

Figure 11.2: Matlab Code to Embed Data in a Cover Image.

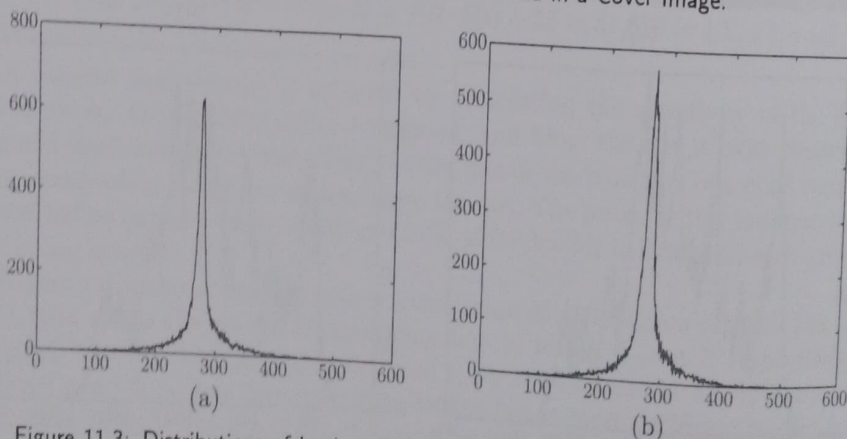


Figure 11.3: Distributions of Laplace-Transformed Pixels of the Lena Image Before and After Embedding.

```
% Matlab file 'LaplaceTransf.m'
% Embed bits of text 'herring' in image 'lena'
filename='lena128'; dim=128;
fid=fopen(filename,'r'); % open cover image
if fid==-1 disp('image file not found'); end;
img=fread(fid,[dim,dim]);
status=fclose(fid);
dummy=LaplaceProc(1,img,dim); % Transform pixels before embed
filename='herring';
fid=fopen(filename,'r'); % open text to be hidden
if fid==-1 disp('data file not found'); end;
[hide,count]=fread(fid,[dim,dim],'ubit1');
count; % input text file bit by bit into 'hide'
status=fclose(fid);
for row=1:dim % double loop to embed 'hide' in
    for col=1:dim % the cover image 'img'
        img(row,col)=bitset(img(row,col),1,hide(row,col));
    end
end
dummy=LaplaceProc(2,img,dim); % Transform pixels after embed
```

```
% Matlab file 'LaplaceProc.m'
function dummy=LaplaceProc(i,img,dim);
dummy=0; min=0; max=0; nimg=zeros(dim,dim);
for col=2:dim-1 % store transformed pixels in
    for row=2:dim-1 % nimg
        nimg(row,col)=img(row-1,col)+img(row+1,col) ...
            +img(row,col-1)+img(row,col+1)-4*img(row,col);
        p=nimg(row,col);
        if p<min % find minimum and maximum
            min=p; % of transformed values
        end;
        if p>max
            max=p;
        end;
    end % for row
end % for col
pdist=zeros(max-min+1,1);
for col=2:dim-1
    for row=2:dim-1
        p=nimg(row,col)-min+1;
        pdist(p)=pdist(p)+1;
    end
end
figure(i), plot(pdist) % dist of transformed pixel values
```

Figure 11.4: Matlab Code for Figure 11.3.

after embedding foreign data. It is clear that the distributions are somewhat different, and that embedding the data has created a “rougher” distribution. It turns out that the difference in pixel distributions can be magnified by transforming the pixels in a special way.

We know from experience that an image tends to have areas that are uniform or close to uniform. When moving from pixel to pixel in such an area, the pixels’ colors don’t vary much. In general, a pixel in an image tends to be similar to its immediate neighbors. If we select the four immediate neighbors of a pixel P and compute their average, the result will, in most cases, be very similar to P . If we subtract the average from P , we can expect to end up with a small number. Such a difference will be large (positive or negative) only when P and some of its neighbors differ much, a fairly rare case. Since the average of integers may be a noninteger, a more practical way of computing the transform is

$$P[x, y] = P[x - 1, y] + P[x + 1, y] + P[x, y - 1] + P[x, y + 1] - 4P[x, y].$$

After the 128×128 pixels of the Lena image are transformed in this way (by the Matlab code of Figure 11.4), their distribution is shown in Figure 11.3a. It is similar to the well-known Laplace distribution, so this transformation is known as a Laplace transform or Laplace filtering.

The point is that changing the pixel values by embedding data (even though only the least significant bits of the pixels are affected) may sometimes change the colors of neighbor pixels enough to affect the Laplace transform significantly. Thus, a good, quick test for a suspect image is to compute the Laplace transform of its pixels and plot the distribution of the transformed pixels (Figure 11.3b). If the distribution is significantly different from a Laplace distribution, the image should be suspect of being a cover (or of having been modified in some way).

The approach described in this section can fail if the cover image is subjected to manipulations such as lossy compression (and decompression), filtering, sharpening, and cropping. This is why *redundancy* is important in steganography. If the same bit is hidden several times in the cover, the decoder has a better chance of recovering it even after the stegoimage has been modified. The Patchwork method of Section 11.6 is a good example of the use of redundancy in steganography.

Steganography—putting encrypted messages in electronic files—is widely used by terrorist groups. A recent government report indicated that terrorists have been hiding pictures and maps of targets in sports chat rooms, on pornographic bulletin boards and on Web sites.

—Jerry Freese

11.1.1 A Color Lookup Table

A modern computer displays an image on a screen by means of pixels. The process starts when a program constructs the image in a buffer, termed the *bitmap*, in memory. Each pixel of the image has an entry in the bitmap, indicating its color. The size of

the entry depends on the number of colors of the image and normally ranges from 1 bit (two colors) to 24 bits ($2^{24} \approx 16.8$ million colors). If the screen resolution is high, the bitmap can become large.

Exercise 11.1: What is the size of the bitmap for a $1K \times 1K \times 24$ image?

A memory buffer of several megabytes is not excessively large, but the point is that an image with a resolution of $1K \times 1K$ has about a million pixels, so it cannot have more than a million different colors. Experience shows that a typical image may have just a few hundred (or at most, a few thousand) colors, which is why an image is normally represented in memory by means of a small bitmap combined with a *color lookup table*.

A typical color lookup table may have 256 rows (allowing for an image with 256 different colors) and three columns (allowing for three bytes or 24 bits to specify each color). Each pixel occupies one byte in the bitmap, so the bitmap size equals the image resolution in bytes. Each entry (byte) in the bitmap contains a pointer to one of the 256 entries of the lookup table (Figure 11.5). The three bytes of the entry are the three components of the color (normally RGB or YCbCr).

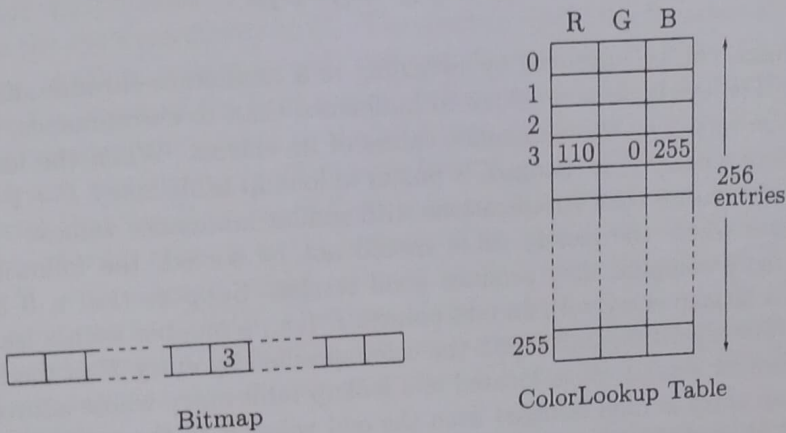


Figure 11.5: A Bitmap and a Color Lookup Table.

Palette—board on which paints are laid and mixed.

When an image is written on a file, both the bitmap and the lookup table have to be included. The well-known GIF and BMP graphics file formats use this convention.

Data can be hidden in such an image file either in the lookup table or in the bitmap. The simplest way to hide data in the lookup table is to store data bits in the least significant positions of the color definitions in the table. In a typical lookup table of size 256×3 , it is possible to hide $256 \cdot 3 = 768$ bits (or 96 bytes), not much. Each group of three bits stored in the least significant bits of three bytes of a table entry slightly modifies the color defined by that entry.

A more sophisticated approach makes use of the fact that the entries of the lookup table don't have to be sorted in any way. If we decide, for example, to swap entries i

and j , all we need to do is scan the bitmap and change all occurrences of i to j and all occurrences of j to i . If the bits to be hidden are $b_1 b_2 b_3 \dots$, we simply move items in the lookup table until the leftmost bit of entry i equals b_i . In a lookup table with 256 entries, it may normally be possible to hide about 200 bits this way. A drawback of this method is that a malicious attacker can easily destroy the hidden data by moving entries around in the lookup table.

Data can also be hidden in the bitmap but the reader should notice that when a color lookup table is used, the bitmap contains pointers and not color specifications. Thus, adjacent bitmap entries may point to lookup table entries with very different colors. Changing the least significant bit of a bitmap entry from, say, 4 to 5, may significantly change the color of the corresponding pixel, because entries 4 and 5 of the lookup table may contain different color specifications.

A simple solution to this problem is to sort the lookup table such that adjacent entries specify similar colors. If each entry specifies the RGB components of a color, then the entries may be sorted according to their Euclidean norm

$$N = \sqrt{R^2 + G^2 + B^2}.$$

This approach can be improved by switching to a luminance-chrominance color representation. The eye is more sensitive to luminance than to chrominance, so the lookup table can be sorted by the luminance values of its entries. When the least significant bit of a bitmap entry P is changed, it points to lookup table entry $P + 1$ or $P - 1$, and these entries contain color specifications with similar luminance values.

In cases where the lookup table should not be sorted, the following approach, although more complex, may produce good results. Suppose that a 0 bit should be hidden in a bitmap entry with an odd pointer P (i.e., a number with a least significant bit of 1). The algorithm starts with the color specified by entry P of the lookup table, and searches for similar colors located in a lookup table entry whose address Q is even. The bitmap entry is then changed from the odd value P to the even value Q , thereby hiding a 0 bit and changing the color by the smallest possible amount.

"It must be that as soon as a culture has reached a certain level, probably measured largely by its literacy, cryptography appears spontaneously—as its parents, language and writing, probably also did. The multiple human needs and desires that demand privacy among two or more people in the midst of social life must inevitably lead to cryptology wherever men thrive and wherever they write. Cultural diffusion seems a less likely explanation for its occurrence in so many areas, many of them distant and isolated." [p. 84]

"It was the amateurs of cryptology who created the species. The professionals, who almost certainly surpassed them in cryptanalytic expertise, concentrated on down-to-earth problems of the systems that were then in use but are now outdated. The amateurs, unfettered to those realities, soared into the empyrean of theory." [pp. 125–126]

—David Kahn, *The Codebreakers* (1967)

Luminance and Chrominance

The most common color space is RGB, where the three color components are the intensities of red, green, and blue. When used in computers, these components are normally in the range 0–255 (8 bits) each. There are, however, other ways to specify color, and several important color spaces use *luminance* as one of the color components. Roughly speaking, luminance corresponds to brightness.

Brightness is the attribute of a visual sensation according to which an area appears to emit more or less light. The brain's perception of brightness is impossible to define although we know from experiments that the eye is most sensitive to green and least sensitive to blue, so blue contributes very little to the brightness that we sense. As a result, a quantity called luminance and denoted by "Y" is used for measurements and calculations in fields such as image processing that are concerned with light, color, and images. Luminance is defined as radiant power weighted by a spectral sensitivity function that is characteristic of vision. In simple words, luminance is a weighted sum of the three color components C_i where each component is multiplied by a weight that's proportional to the eye's sensitivity to C_i . The spectral sensitivity function assigns large weight to green, medium weight to red, and very little weight to blue. Luminance is proportional to the power of the light source. It is similar to intensity, but the spectral composition of luminance is related to the brightness sensitivity of human vision.

The eye is very sensitive to small changes in luminance, which is why it is useful to have color spaces that use Y as one of their three components. A simple way to define a luminance-based color space is to (1) compute Y as a weighted sum of the intensities of R, G, and B, with a small weight assigned to blue, then (2) subtract Y from the blue and red components of RGB. The three components Y, $B - Y$, and $R - Y$ become a new color space termed YCbCr. The last two components are called chroma. They represent color in terms of the presence or absence of blue (Cb) and red (Cr) for a given luminance intensity.

The YCbCr color space is now precisely defined by an international standard. Y is defined to have a range of 16 to 235; Cb and Cr are defined to have a range of 16 to 240, with a value of 128 in the case where R, G, and B are all zero.

Conversions between RGB with a 16–235 range and YCbCr are linear and therefore simple. Transforming RGB to YCbCr is done by (notice the small weight of blue):

$$\begin{aligned} Y &= (77/256)R + (150/256)G + (29/256)B, \\ Cb &= -(44/256)R - (87/256)G + (131/256)B + 128, \\ Cr &= (131/256)R - (110/256)G - (21/256)B + 128; \end{aligned}$$

while the opposite transformation is

$$\begin{aligned} R &= Y + 1.371(Cr - 128), \\ G &= Y - 0.698(Cr - 128) - 0.336(Cb - 128), \\ B &= Y + 1.732(Cb - 128). \end{aligned}$$

11.2 BPCS Steganography

BPCS (bit-plane complexity segmentation) is the brainchild of Eiji Kawaguchi. Developed in 1997, this steganography method hides data in a cover image (color or grayscale) and its main feature is large hiding capacity [BPCS 01]. The size of the hidden data is typically 50–70% of the size of the cover image, and hiding the data does not increase the size of the cover image.

Figure 11.6 shows a grayscale image of parrots, together with five of its bitplanes (bitplane 8, the most significant one, bitplane 1, the least significant one, and bitplanes 3, 5, and 7). It is clear that the most significant bitplane is somewhat similar to the full image, the least significant bitplane is random (or at least seems random) and, in general, as we move from the most significant bitplanes to least significant ones, they become more random. However, each bitplane has some parts that look random. (BPCS converts the pixels of the cover image from binary to *Gray codes*, which are discussed in Section 11.2.1. When an image is represented by Gray codes, there is less difference between the least significant and most significant bitplanes; the randomness, or redundancy, in the image is distributed more uniformly.)

The principle of BPCS is to separate the image into individual bitplanes, check every 8×8 -bit square region in each bitplane for randomness, and replace each random region with 64 bits of hidden data. Regions that are not random (*shape-informative* regions in BPCS terminology) are not modified. A special complexity measure α is used to determine whether a region is random. A color image where each pixel is represented by 24 bits has 24 bitplanes (8 bitplanes per color component). A grayscale image with 8 bits/pixel has eight bitplanes. The complexity measure α of an 8×8 block of bits is defined as the number of adjacent bits that are different. This measure (see note below) is normalized by dividing it by the maximum number of adjacent bits that can be different, so α is in the range $[0, 1]$.

An important feature of BPCS is that there is no need to identify those regions of the cover image that have been replaced with hidden data, because the hidden data themselves are transformed, before they are hidden, to a random representation. The BPCS decoder identifies random regions by performing the same tests as the encoder, and it extracts the 64 data bits of each random region. The hidden data are made to look random by first compressing them. It is known (see, for example, [Salomon 00]) that compressed data have little or no redundancy, and therefore look random. If a block of 64 bits (after compression) does not pass the BPCS randomness test, it goes through a *conjugation operation* that increases its randomness. The conjugation operation computes the exclusive-OR of the 64-bit block (which is regrouped as an 8×8 square) with an 8×8 block that has a checkerboard pattern and whose upper-left corner is white. This transforms a simple pattern to a complex one and changes the complexity measure of the block from α to $1 - \alpha$. The encoder has to keep a list of all the blocks that have gone through the conjugation operation, and this list (known as the conjugation map) is also hidden in the cover image as shown below.

Figure 11.7 shows how the BPCS image complexity measure α is defined. Part (a) of the figure shows a 4×4 image with the 16 pixels (1 bit each) numbered. The complexity of the image depends on how many adjacent bits differ. The first step is to



Figure 11.6: Bitplanes 1, 3, 5, 7, and 8 of the Parrots Image.

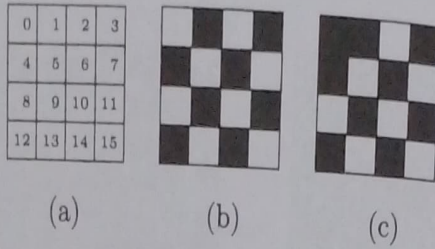


Figure 11.7: BPCS Image Complexity Measure.

compare each bit in the top row (row 1) to its neighbor below and count how many pairs differ. Then the four pairs of bits of rows 2 and 3 are compared, followed by the pairs in rows 3 and 4. The maximum number of different pairs is 4×3 . The next step is to compare adjacent bits horizontally, starting with the four pairs in columns 1 and 2. This can also yield a maximum of 4×3 different bit pairs. The image complexity measure α is defined as the actual number of bit pairs that differ, divided by the maximum number, which for a 4×4 block is $2 \cdot 4(4 - 1) = 24$ and for an 8×8 block is $2 \cdot 8(8 - 1) = 112$.

Figure 11.7b shows a checkerboard pattern, where every pair of adjacent bits differs. The value of α for such a block is 1. Figure 11.7c shows a slightly different block, where the value of α is

$$\alpha = \frac{(3 + 4 + 4) + (3 + 4 + 4)}{2 \cdot 4 \cdot 3} = 0.917.$$

BPCS considers a block random if its complexity measure α is greater than or equal to a certain threshold. Experiments indicate that a threshold value of about 0.3 provides large embedding capacity without noticeably degrading the visual quality of the cover image. (The value 0.3 also equals $0.5 - 4\sigma$, where $\sigma = 0.047$ is a constant. The value of σ was determined by computing α values for many 8×8 blocks and plotting them. The distribution of the α values was found to be Gaussian with mean 0.5 and standard deviation 0.047.)

The simplest way to organize the conjugation map is to include in it one bit for each block of secret data. However, if the complexity measure α of a data block is in the interval $[\text{threshold}, 1 - \text{threshold}]$, then the block does not have to be conjugated and the BPCS decoder *can figure out this fact*. The conjugation map does not have to have a bit for such a data block. If the map is organized this way, the decoder has to compute the complexity measures α of each data block and its conjugate. If both are in the interval $[\text{threshold}, 1 - \text{threshold}]$, then the decoder knows that the block is not conjugated and the conjugation map has no bit for the block. This can reduce the size of the map significantly.

The last point to consider is how to hide the conjugation map itself in the image without increasing the image size. The map is hidden just like the rest of the hidden data, each block of conjugation map is embedded in a random image block. However, some blocks of the conjugation map may not be random and may have to be conjugated. To avoid having a conjugation map of the conjugation map, we can place a conjugation indicator inside each block of the conjugation map. Each 8×8 block of the conjugation map contains 64 bits of which only 63 are used for conjugation information and the remaining one is used as an indicator. The indicator is a bit located in a position that

gets complemented during conjugation. The bit is set to zero before the conjugation block is hidden. If the block ends up being conjugated, the bit becomes a 1. The decoder knows what blocks make up the conjugation map, and it conjugates those that have an indicator bit of 1 (and then ignores or deletes the indicator bit). Note that the entire conjugation map can be prepared while the secret data is being embedded in the image, because the map depends on the data, not on the image. Once prepared, the conjugation map can be embedded in the image, following the blocks of the secret data; ingenious!

11.2.1 Gray Codes

We are familiar with the standard binary representation of integers, where the value of a bit depends on its position in the number. Thus, the binary integer 1101_2 equals $2^3 + 2^2 + 2^0 = 13$. This is similar to the well-known decimal representation and makes it easy to operate on binary integers. However, there are other ways to represent integers with bits, and one of them, namely Gray codes, has advantages for the representation of images.

Imagine a grayscale image where each pixel is represented by four bits $b_3b_2b_1b_0$. Thus, a pixel can have one of 16 grayscales. We now separate the image into four bitplanes, where bitplane i consists of bits b_i of all the pixels. What can we say about the four bitplanes? In principle, they can be anything, but in practice there is a difference between them because of the nonrandom nature of digital images. An image that we store in our computers is not random. Random images are generally useless, uninteresting, and unimportant; we don't keep such an image. If an image is important and interesting, then it is nonrandom and therefore has an important property. Neighboring pixels in such an image tend to be similar. If we select a pixel at random in an image, chances are that its immediate neighbors will be similar to it, and even its near (not immediate) neighbors will mostly be similar. We say that the pixels of an image are *correlated*.

When the image is separated into individual bitplanes, we find, however, that the least significant bitplanes are random, or almost so, while the most significant bitplanes look like black-and-white versions of the original image. The following experiment illustrates this behavior. We write the 16 4-bit integers 0–15 right next to each other. The difference between any of these numbers and its immediate neighbors is 1.

1111 1110 1101 1100 1011 1010 1001 1000 0111 0110 0101 0100 0011 0010 0001 0000.

When we separate these numbers into four "bitplanes," we immediately see the difference between them. The bits of the leftmost bitplane are highly correlated, they change only once. The bits of the rightmost (least significant) bitplane, on the other hand, alternate all the time

1111111100000000, 1111000011110000, 1100110011001100, 1010101010101010.

This behavior of the bitplanes is a result of a basic feature of binary numbers. Consecutive binary numbers always differ in their least significant bits, but rarely differ in their

most significant bits. If we want to separate an image into bitplanes that are similar, we should use a different binary representation for the pixels.

The Gray binary representation, known as Gray codes or reflected Gray codes (RGC), has the advantage that the codes of consecutive integers differ by exactly one bit.

This code is easy to generate with the following recursive construction (see also [Press and Flannery 88]). Start with the two 1-bit codes (0, 1). Construct two sets of 2-bit codes by duplicating (0, 1) and appending, either on the left or on the right, first a zero, then a one, to the original set.

The result is (00, 01) and (10, 11). Now reverse (reflect) the second set, and concatenate the two. The result is the 2-bit RGC (00, 01, 11, 10); a binary code of the integers 0 through 3 where consecutive codes differ by exactly one bit. Applying the rule again produces the two sets (000, 001, 011, 010) and (110, 111, 101, 100), which are concatenated to form the 3-bit RGC. Note that the first and last codes of any RGC also differ by one bit.

Here are the first three steps for computing the 4-bit RGC:

Add a zero (0000, 0001, 0011, 0010, 0110, 0111, 0101, 0100),
Add a one (1000, 1001, 1011, 1010, 1110, 1111, 1101, 1100),
reflect (1100, 1101, 1111, 1110, 1010, 1011, 1001, 1000).

Table 11.8 shows how individual bits change when moving through the binary codes of the first 32 integers. The 5-bit binary codes of these integers are listed in the odd-numbered columns of the table, with the bits of integer i that differ from those of $i - 1$ shown in boldface. It is easy to see that the least significant bit (bit b_0) changes all the time, bit b_1 changes for every other number, and, in general, bit b_k changes every k integers. The even-numbered columns list one of the several possible reflected Gray codes for these integers. The table also lists a recursive Matlab function to compute RGC.

<u>43210</u>	<u>Gray</u>	<u>43210</u>	<u>Gray</u>	<u>43210</u>	<u>Gray</u>	<u>43210</u>	<u>Gray</u>
00000	00000	0 1 000	10010	1 0000	00011	1 1000	10001
00001	00100	01001	10110	10001	00111	11001	10101
000 1 0	01100	010 1 0	11110	100 1 0	01111	110 1 0	11101
00011	01000	01011	11010	10011	01011	11011	11001
00 1 00	11000	01 1 00	01010	10 1 00	11011	11 1 00	01001
00101	11100	01101	01110	10101	11111	11101	01101
001 1 0	10100	011 1 0	00110	101 1 0	10111	111 1 0	00101
00111	10000	01111	00010	10111	10011	11111	00001

```
function b=rgc(a,i)
[r,c]=size(a);
b=[zeros(r,1),a; ones(r,1),flipud(a)];
if i>1, b=rgc(b,i-1); end;
```

Table 11.8: First 32 Binary and Reflected Gray Codes.

11.3 Lossless Data Hiding

The data hiding methods described so far distort the coverimage permanently. The distortion can be made imperceptible to the eye, but it is still present in the stegoimage. Following the terminology used in data compression, such steganographic methods can be termed lossy or irreversible. In most applications, the coverimage is irrelevant to both sender and receiver (encoder and decoder), so the stegoimage is discarded by the receiver after the hidden data have been extracted. In certain applications, however, the coverimage is important and the receiver would like to restore the stegoimage to the original coverimage after the hidden data are extracted. A data hiding method that allows a complete reconstruction of the coverimage is referred to as lossless (or reversible, invertible, or distortion-free).

A lossless steganographic algorithm can be implemented in the following steps: (1) Identify the part of the coverimage that's going to be modified by the data-embedding process. (2) Compress this part. (3) Append the compressed part to the secret data. (4) Embed the secret data and the compressed part. Decoding is the exact opposite of encoding. After the hidden data are recovered, the compressed part is extracted, decompressed, and used to replace that part of the stegoimage that had been modified by the encoder. This results in the original, unmodified coverimage.

Following are three situations where lossless data hiding is important.

1. Medical applications. A watermark is hidden in an X-ray image. The watermark may consist of the names of the patient, doctor, and hospital, as well as confidential medical information. In an X-ray image, any pixel may be significant, so it is important to completely reconstruct the image after the watermark has been extracted.
2. An image taken by a spy satellite is examined by experts by filtering or under conditions of high zoom. Many times, individual pixels are scrutinized and may be the source of important information or may provide important clues. It makes sense to watermark such an image (with the date the image was taken, the place shown in the image, the date it was examined, the name of the examiner, and various notes) and this kind of watermarking should obviously be lossless.
3. Artists displaying and selling images over the Internet want to watermark their images as protection against theft and as a means to legally identify pirated images. Some artists are picky and object to even a single pixel being modified in their images, even when the modifications are visually imperceptible. Such artists can also benefit from lossless data embedding.

The main problem facing a lossless data hiding algorithm is that the secret data are embedded in an image component that's visually unimportant, such as the LSB (least significant bit) of certain pixels. Because of its unimportance, such a component is normally random or close to random and therefore hard or impossible to compress. A lossless steganographic algorithm should therefore be more sophisticated than simple LSB embedding.

We start with two lossless data embedding methods for raw images. Section 11.3.1 describes methods for data embedding in JPEG images and Section 11.3.2 describes methods for data embedding in GIF images. The methods presented here follow the work described in [Fridrich et al. 02].

The first method described here is called “RS lossless data embedding.” It starts with a coverimage V and it is based on selecting a component (or a part or a feature) B of V . B is extracted, losslessly compressed, concatenated with the secret data, and the result embedded in V , replacing the original B . In order for this method to work, component B has to satisfy the following. (1) It is compressible (i.e., is not random) and (2) it can be modified and replaced by other data without affecting important image features. If B satisfies these two requirements, then decoding the image is the opposite of encoding and restores image V to its original form.

Here is this method in more detail. We assume that image V has $M \times N$ pixels, each with values in a certain set P [for example, $P = (0, 1, \dots, 255)$]. The image is partitioned into groups G of n adjacent pixels $p_1, p_2, \dots, p_n$ each. Examples of such groups are n pixels in a row, n pixels in a column, and a square consisting of n pixels. The number of groups is $M \times N/n$. A *discrimination function* $f(G)$ is defined for the groups $G = (p_1, p_2, \dots, p_n)$ in order to measure the smoothness or regularity of the group members. An example of such a function is

$$f(p_1, p_2, \dots, p_n) = \sum_{i=1}^{n-1} |p_{i+1} - p_i|,$$

but other functions are possible. The next step is to define an involutory “flipping” permutation F on the set P of pixel values. [An involutory permutation is a permutation that’s its own inverse (Section 2.2).] For example, F may be the permutation $0 \leftrightarrow 1$, $2 \leftrightarrow 3$, up to $254 \leftrightarrow 255$. If we consider P a set of grayscale values, then this particular F corresponds to flipping the LSBs of these values. We can therefore denote it by F_{LSB} and say that it has an amplitude of 1.

- ◇ **Exercise 11.2:** An involutory permutation that flips the values of the second LSB can be considered to have a bigger “amplitude.” What is this permutation?

There may be flipping permutations with bigger amplitudes and also permutations where more than one bit is flipped. Thus, we define the *amplitude* A of a given flipping permutation F as the average change of values induced by applying F ,

$$A = \frac{1}{|P|} \sum_{v \in P} |v - F(v)|.$$

For the F_{LSB} permutation, each difference $|v - F(v)|$ is 1, so A is always 1, regardless of the size of P . For the permutation of Exercise 11.2, A is 2.

The discrimination function f and the flipping permutation F are used to define the three types R , S , and U (for regular, singular, and unusable) of pixel groups as follows: Group G is of type R if it satisfies $f(F(G)) > f(G)$. Similarly, S is the type of all groups G that satisfy $f(F(G)) < f(G)$, and U is the type of all the groups for which $f(F(G)) = f(G)$.

In a typical image, adjacent pixels are correlated. We therefore expect $f(G)$ to be small for most groups G . Once permutation F has been applied to the pixels of a group G , they (the pixels) become perturbed and therefore less correlated. When the

discrimination function f is applied to $F(G)$ it therefore results in a larger value. This is why most groups in an image are of type R .

We denote by N_R , N_S , and N_U the numbers of groups of type R , S , and U , respectively. Thus, $N_R + N_S + N_U = M \times N/n$ and for "regular" images $N_R > N_S$. It is this difference between the numbers of R and S groups that's exploited to hide the secret data. This is also the reason why this method is called RS lossless data embedding. Notice that the following relations hold:

If G is of type R , $F(G)$ is of type S .

If G is of type S , $F(G)$ is of type R .

If G is of type U , $F(G)$ is of type U .

These relations can be denoted by $F(R) = S$, $F(S) = R$, and $F(U) = U$. We can also say that the flipping permutation F flips the two types R and S but leaves type U unchanged.

We are now ready for the details of RS lossless embedding. The image is scanned in a certain order and an RS bitstring is constructed. For each group of type R encountered in the scan, a 0 is appended to RS, and for each type- S group, a 1 is appended to it. Groups of type U are rare and do not participate in the data embedding (that's done later). Such groups are simply skipped and do not affect RS. At the end of the scan, RS is complete, its size is $N_R + N_S$, and it is compressed. RS compresses well because each R group contributes a 1 and each S group contributes a 0 to it and there are more R than S groups. Thus, RS has more 1s than 0s, so it is *sparse* (or close to sparse) and sparse binary strings can be highly compressed (see, for example, [Salomon 00], page 704).

The secret data to be embedded are now appended to RS and the extended RS is embedded in the R and S groups. (The extended RS string must start with a count or a pointer, to enable the decoder to break it up into its two constituents.)

Embedding the extended RS string is done by scanning the image as before, looking for R and S groups. When the next such group is encountered, the next bit from the extended RS string is embedded in the group. If the group is of type R and the next bit is 1, no change is needed. If the next group G is of type R and the next bit is 0, the group is flipped to $F(G)$, whose type is S . This process can be summarized by saying that type- R groups are assigned to secret bits 1 and type- S groups are assigned to secret bits 0. If the next group and the next bit are mismatched, the group is flipped.

Decoding is simple. The stegoimage is scanned in the same order and a bitstring is generated by appending a 1 bit for each R -type group of pixels encountered and a 0 bit for each group of type S . The string is then broken up into the compressed RS string and the secret data. The RS string is decompressed and is used to reconstruct the original image. This is done by scanning the image again, in the same order, looking for groups of types R and S . Each such group encountered corresponds to the next bit in RS. If the group type and the bit are mismatched, the group is flipped.

Since bits are embedded in the R and S groups, the total number of bits that can be embedded (the raw data capacity of the image) is $N_R + N_S = M \times N/n - N_U$ bits. If we denote the size of the compressed RS string by $|RS|$, then the net data capacity (the payload) is $D = N_R + N_S - |RS|$. The magnitude of $|RS|$ depends on the efficiency of the compression method used and also on the difference between R and S . If R and

S are about equal, then string RS is not sparse and may be close to random; it will not compress well. If, on the other hand, there is a big difference between R and S , the string RS (whose size is $N_R + N_S$) will compress well. The best possible compression (the so-called *entropy compression*) will compress RS to

$$-N_R \log_2 \left(\frac{N_R}{N_R + N_S} \right) - N_S \log_2 \left(\frac{N_S}{N_R + N_S} \right) \quad \text{bits.}$$

The payload is therefore

$$D = N_R + N_S + N_R \log_2 \left(\frac{N_R}{N_R + N_S} \right) + N_S \log_2 \left(\frac{N_S}{N_R + N_S} \right) \quad \text{bits.}$$

It is easy to see that $N_R = N_S$ implies $D = 0$, while a big difference between N_R and N_S results in large D . The difference between N_R and N_S depends on the size and shape of the pixel groups G , on the discrimination function f , on the amplitude A of the flipping permutation F , and on the nature of the original image V . The difference increases with large group size n and with large amplitudes. Also, smooth, quiet images tend to result in bigger differences and therefore in increased payload capacity.

Experiments performed by the originators of this method on various images (all using $n = 4$) indicate that the number of secret bits embedded on average in one image pixel varies from 0.019 (for flipping permutation with amplitude 1) to 0.12 (for amplitude 7). Another result was that even the small group size $n = 4$ and the amplitude 1 F_{LSB} permutation performed well on many images.

The next method described in this section is based on the idea of locating in image V subsets (of pixels) with specific properties. We denote by $S(x)$ the set of all pixels in V whose value is x . We start by identifying two sets $S(x)$ and $S(y)$ that satisfy the following conditions.

1. The values x and y are similar.
2. The two sets $S(x)$ and $S(y)$ can be compressed; i.e., the pixels that constitute those sets are correlated.

Image V is scanned in some order (perhaps rowwise) and a binary string Z is prepared. For each x pixel found during the scan, a 0-bit is appended to Z . For each y pixel found, a 1-bit is similarly appended. Bitstring Z is then losslessly compressed and concatenated with the secret data to form bitstring W . Notice that W must start with a tag or a pointer that will make it possible for the decoder to identify its two parts (string Z and the secret data). The last step is to embed W in V by modifying some of the x and y pixels. Embedding is done by hiding each 0 bit of W as an x pixel and each 1 bit as a y pixel. If the next bit to be hidden is a 0, the algorithm scans the image until the next x or y pixel is encountered. If that pixel is y , it is changed to an x . If that bit is an x , there is no need for any changes.

Notice that bitstring Z is compressible if the sizes of sets $S(x)$ and $S(y)$ are very different. In such a case, Z has either many 0s and a few 1s or vice versa. In either case, Z would be sparse and therefore highly compressible.

Decoding is done by first extracting W from the stegoimage. The image is scanned in the same order as in encoding. Each x pixel encountered appends a 0 to the extracted bitstring and each y pixel encountered appends a 1 to it. The resulting string W is then broken up into the secret data and bitstring Z . The latter is decompressed and is used to restore the original x and y pixels. This is done by scanning the image, looking for x or y pixels. When such a pixel is encountered, the next bit of Z determines whether its value should be switched.

An important feature of the method is that changing x pixels into y and vice versa does not affect the image much because we require the x and y values to be close.

This approach can be used for raw or compressed images, but has proved effective for palette images, of which the GIF format is a well-known example. (Section 11.3.2 describes other lossless data hiding methods better suited for palette images.)

11.3.1 Lossless Data Hiding in JPEG Images

The two methods described in this section have been designed specifically to embed data losslessly in JPEG images. Given a coverimage in JPEG format, the encoder embeds a payload in it (secret data plus compressed information from the coverimage), producing a JPEG stegoimage. The decoder extracts the payload and restores the stegoimage to the original JPEG coverimage.

Both methods embed the secret data in the quantized DCT coefficients of the coverimage in such a way that the decoder can later reconstruct the original JPEG coverimage from the watermarked JPEG stegoimage without any loss of data. The JPEG image compression method is discussed in detail in [Pennebaker and Mitchell 92] and [Salomon 00]. The reader should be familiar with the basic steps of JPEG compression (they are listed in this book, in Section 11.9) before reading the remainder of this section.

The first method described here starts with a given JPEG coverimage. The image is partitioned into b blocks of 64×64 pixels each, and the same position (i, j) is selected in all the blocks. The secret data are embedded in the LSBs of the DCT coefficients located at that position. If more data remain to be hidden, another position is selected and the embedding process is continued. We denote by $d_k(i, j)$ the DCT coefficient at position (i, j) in block k , where $0 \leq i, j \leq 7$, and $k = 1, 2, \dots, b$. We further denote by $D_k(i, j)$ the quantized value of $d_k(i, j)$. Quantization is done by the division

$$D_k(i, j) = \frac{d_k(i, j)}{Q(i, j)},$$

where $Q(i, j)$ are quantization coefficients that depend on i and j (they are larger for larger values of i and j), but don't depend on the block. The quantization coefficients can be supplied by the user, but there are default values recommended by the JPEG standard, and those are normally used.

Because of the properties of the DCT, most of the DCT coefficients are small, so most of the quantized coefficients become 0, 1, or -1 . The only large quantized coefficients are those for small i and j . The sequence $D_k(6, 5)$, for example, consists mostly of 1s and -1 s with runs of many zeros separating them. The sequence is therefore highly

compressible. The lossless method selects a position (i, j) , constructs the string consisting of the b LSBs of $D_k(i, j)$, and compresses this string losslessly. The compressed string with the secret data appended to it becomes the payload. It is embedded in the LSBs of all the $D_k(i, j)$ s.

In order to improve the compression of the LSBs, the number of zeros in the string of LSBs is increased (and the number of nonzeros is correspondingly decreased) by defining the LSB of a negative D_k as $\text{LSB}(D_k) = 1 - (|D_k| \bmod 2)$. Thus, the LSBs of negative odd D_k s become 1 and those of negative even D_k s become 0. The decoder looks at the sign of D_k before it interprets the LSB, so it can always interpret the LSB correctly.

Large values of i and j correspond to large quantization coefficients, so they result in a sequence of quantized coefficients $D_k(i, j)$ with more zeros and fewer 1s and -1s. Such a sequence compresses better, but embedding data in it results in higher distortion of the stegoimage after the payload is embedded and the coverimage is JPEG compressed and viewed. The values of i and j should therefore correspond to DCT coefficients of medium frequencies. Both small values (such as 1 or 2) and large values (such as 6 or 7) should be avoided. The developers of the method also recommend to embed the data in the chrominance components of the image because the eye is less sensitive to changes in chrominance.

Here are the details of this method.

1. The user determines a value l and l pairs of medium-frequency positions $(i_1, j_1), (i_2, j_2), \dots, (i_l, j_l)$.
2. The JPEG file is input and the Huffman codes in it are used to compute the values of the quantized coefficients $D_k(i, j)$ for all values of i, j , and k .
3. A sequence S of $l \times b$ quantized coefficients is prepared as follows:

$$D_1(i_1, j_1), D_1(i_2, j_2), \dots, D_1(i_l, j_l), D_2(i_1, j_1), D_2(i_2, j_2), \dots, D_2(i_l, j_l), \dots, D_b(i_1, j_1), D_b(i_2, j_2), \dots, D_b(i_l, j_l).$$

4. A pseudo-random number generator is used to select coefficients from S at random. The LSB of each coefficient selected in this way is appended to a string B and B is entropy-compressed all the time to a string C . This continues until the difference (number of visited coefficients - size of C) equals the size of the secret data. The set of visited coefficients is denoted by S_1 .

5. The compressed string C is concatenated to the secret data and the resulting bitstring (the payload) is embedded in the LSBs of the coefficients of S_1 . The length of the payload is also embedded, for the decoder's use. All the quantized coefficients visited in Step 2 are now compressed with the Huffman codes recommended by JPEG, thereby resulting in a new, watermarked JPEG stegoimage.

The decoder proceeds in similar steps as follows.

1. The user inputs l and the l pairs of medium-frequency positions.
2. The watermarked JPEG stegoimage is input and the quantized coefficients are computed by decompressing the Huffman codes.

3. Sequence S is determined as with the encoder and the same pseudo-random number generator is used to visit coefficients in it.

4. The LSBs of the selected coefficients are appended to a string C , which is decompressed all the time to a string B . Once the length of B reaches $(b + \text{payload size})$, start the next step.

5. The secret data are cut off from string B and the remaining bits are used to restore the LSBs of the visited coefficients (paying special attention to the negative coefficients). The image is then compressed to what it was before any data were embedded.

The second method of this section is based on the fact that the LSB of an even number is 0 and the LSB of an odd number is 1. The encoder starts by reading the JPEG image file, decompressing it to obtain the DCT coefficients, and computing the quantized coefficients. For each coefficient $D_k(i, j)$, its quantization coefficient $Q(i, j)$ is examined. If $Q(i, j)$ is even, it is divided by 2 (without any loss of bits) and all the quantized coefficients $D_k(i, j)$ that it divides are multiplied by 2. Those quantized coefficients now have a LSB of 0, so bits of the payload can be embedded in them and later extracted trivially, without having to store any special information in the payload. If $Q(i, j)$ is odd, it is replaced by $\lfloor Q(i, j)/2 \rfloor$ (thereby losing its LSB) and all the quantized coefficients $D_k(i, j)$ that it divides are multiplied by 2. Those quantized coefficients now also have a LSB of 0, so bits of the payload can be embedded in them. In this case, however, a flag consisting of the values of i and j has to be included in the payload. This flag is later used by the decoder to identify the odd quantization coefficients.

A drawback of this method is that the table of quantization coefficients has been modified, so the new table has to be included in the compressed stegoimage. The JPEG standard allows the inclusion of nonstandard quantization tables, but the presence of such a table may make the stegoimage suspect. Thus, this method is steganographically weak.

It is possible to have several variations of this method. For example, instead of replacing $Q(i, j)$ by half of itself, it can be replaced by 1, and each visited $D_k(i, j)$ multiplied by the original $Q(i, j)$. The original $Q(i, j)$ should also be included in the payload, to enable the decoder to reconstruct the original JPEG image. The task of the decoder is simple. It has to check whether each DCT coefficient is a multiple of $Q(i, j)$. This variant results in a bigger stegoimage because the DCT coefficients used to embed the payload (the members of sequence S_1) are not quantized. On the other hand, the lack of quantization results in less distortion when the stegoimage is viewed.

11.3.2 Lossless Data Hiding in GIF Images

GIF (Graphics Interchange Format) is a graphics file format especially suited for images with a small number of colors. It uses the concept of a color lookup table (or palette; see Section 11.1.1). The table has 256 entries, each containing the three components of a color. Each image pixel becomes a pointer to this table. The sequence of pointers is then compressed with the well-known LZW dictionary-based compression method [Salomon 00]. A more detailed description of the GIF format is easily found on the World Wide Web. This format lends itself naturally to lossless data embedding. We discuss two versions.

The first version is for images that have fewer than 256 colors. The encoder proceeds in the following steps.

1. It decompresses the list of pointers, and scans the image row by row, counting the number of times each color appears (this results in the histogram of the image). The encoder then finds the most common color appearing in the image. Let's denote this color by d . (If there is more than one most common color, the first one to occur in the image becomes d .)
2. The next unused palette entry is initialized to color d .
3. There are now two palette entries, i and j , with color d . They are associated with 0 and 1, respectively.
4. The image is scanned again in the same order and the secret bits are embedded in the pixels of color d . If the next bit to be hidden is 0, the next pointer to color d is changed (if needed) to i . Similarly, if the next secret bit is 1, the next pointer to color d is changed (if needed) to j .
5. The new list of pointers is compressed.

It is obvious that this method is lossless and no distortion is created by embedding the secret data.

The embedding capacity is the number of pixels of color d . It is possible to extend the capacity by selecting the next common color. If that color is located in entry k of the palette, a new entry, l can be initialized with that color. Instead of embedding single bits, the encoder now embeds pairs of bits. The next pair of secret bits to be hidden can be 00, 01, 10, or 11. These pairs can be associated with pointers i , j , k , and l , respectively. Naturally, this idea can be extended to more than two colors. If the most common C colors are used to embed secret data in the coverimage, then the value of C has to be somehow transmitted to the decoder. One way to do this is to store C in the first entry (entry 0) of the color palette, move the color originally stored in that entry to an unused entry, and modify all the pointers that point to entry 0 to point to the new entry. All this can be done if there are enough unused entries in the palette.

When the list of new pointers is compressed, in Step 5 above, the result may be bigger than the original, because replacing some i pointers with j pointers increases the randomness of the list of pointers.

The decoder starts by decompressing the list of pointers. It then scans the image in the same order as the encoder, looking for pointers i and j and generating 0 and 1 bits accordingly. There is no need to reconstruct the original image because the pixels of the stegoimage are identical to those of the original coverimage. A careful decoder may want, however, to change all j pointers back to i pointers and delete entry j from the palette. This wipes out any traces of the hidden data.

Notice that the decoder has to find out what pointers i and j have been used by the encoder and this has to be accomplished before any secret bits can be extracted. The obvious way to do this is to compute the image histogram in the same way as the encoder and select the most common color.

If entry 0 of the palette indicates the number of colors used to embed the secret data, the decoder has to first examine that entry, then determine from the histogram the C most common colors and their positions in the palette (each must appear twice).

The second version is for use in GIF (or other palette) images where all 256 colors are used. The encoder proceeds as follows.

1. For each pointer i , we denote by $S(i)$ the set of pixels that point to entry i of the palette. We denote the color stored in entry i by c_i . The encoder selects two sets $S(i)$ and $S(j)$ such that colors c_i and c_j are similar, but the sets have very different sizes. If the RGB color components are used in the palette, then we can define the distance between two colors as the sum $|R_i - R_j| + |G_i - G_j| + |B_i - B_j|$.
2. Once colors c_i and c_j have been selected, the encoder moves them to the top two entries of the palette by swapping entries. This also necessitates changing pointers in the image. If color c_i is moved from entry i to entry 0, then all the pointers i have to be changed to 0 and all the pointers 0 should be changed to i .
3. The encoder scans the image in a certain order, such as row by row, locating all the pointers i and j . For each pointer i located, a 0 is appended to an (initially empty) bitstring B . For each pointer j , a 1 is appended to B . The resulting B is sparse, since the sizes of $S(i)$ and $S(j)$ are very different, so it compresses efficiently to a bitstring C , which is concatenated with the secret data to form the payload.
4. The payload is embedded in the coverimage by scanning the image, again looking for pointers i and j . If the next pointer is i and the next bit to be hidden is 0, then no change is needed and similarly, if the next pointer is j and the next bit is 1. If the pointer type and the bit are mismatched, the pointer is flipped from i to j or vice versa. Some distortion is introduced by this flipping but not much because the two colors are similar.

An interesting point to mention is that an image with large uniform areas or with large structures can be used to embed data in this method even if the sizes of the two sets don't differ much. In such an image, string B will have runs of 0s and 1s, so it will also be compressible, although the LZW method seems to perform better in such cases than an entropy compressor such as arithmetic coding.

The decoder finds colors c_i and c_j in the first two entries of the palette. It scans the image in the same order as the encoder and extracts the hidden bits from pointers i and j . It then breaks the resulting string up into bitstring C and the hidden data. String C is decompressed into string B , which is then used to restore the original values of those i and j pointers that had been modified by the encoder.

The data hiding capacity of this version can be increased by using more than two colors.

Jessica Fridrich, the principal originator of the lossless methods described here, specializes in all aspects of information hiding in digital imagery, including watermarking for authentication and tamper detection, self-embedding, robust watermarking, steganography and steganalysis, forensic analysis of digital images (detection of forgeries), advanced image processing, and encryption techniques. Her Web site is at URL
<http://www.ssie.binghamton.edu/fridrich/>.